

CONTROL CHECK AI

Product Requirements Document v0.3

Version 0.3 | Backend Persistence Alignment | 17 August 2026

AI-Powered Project Control Assurance Platform

Product	ControlCheck AI
Version	0.3
Status	Development Blueprint
Primary Market	EPC / Construction / Oil & Gas Contractors
Primary Users	Project Control, Project Managers, Management
Frontend	React + TypeScript + Tailwind + shadcn/ui
Backend	FastAPI + Python
Database	PostgreSQL
AI	LLM + deterministic tool/data layer
Analytics	Python / Pandas / NumPy / Scikit-learn
First Release Goal	Upload project data and produce evidence-backed findings

Confidential working document. This PRD is intended to guide product, UX, engineering, data, and AI implementation for the first sellable version of ControlCheck AI.

Table of Contents

- 1. Executive Summary
- 2. Product Vision & Strategy
- 3. Problem Statement
- 4. Target Market & Personas
- 5. Goals, Non-Goals & Success Metrics
- 6. Product Scope & Feature Matrix
- 7. User Journeys
- 8. Information Architecture & UX
- 9. Data Model & Canonical Data Contract
- 10. Data Ingestion & Quality Engine
- 11. Control Rule Engine
- 12. Findings & Evidence Engine
- 13. Project Health Scoring
- 14. AI Intelligence Architecture
- 15. AI Assistant Requirements
- 16. API Specification
- 17. Security, Multi-Tenancy & Governance
- 18. Reporting
- 19. Technical Architecture
- 20. MVP Roadmap & Delivery Plan

- 21. QA, Evaluation & Acceptance Criteria
- 22. Pilot & Go-to-Market
- 23. Pricing Hypothesis
- 24. Risks & Mitigations
- 25. Definition of Done
- Appendix A: Example Rules
- Appendix B: Example API Payloads
- Appendix C: Demo Scenario

1. Executive Summary

ControlCheck AI is a SaaS platform that acts as an AI-powered project control assurance layer. It ingests project data from Excel/CSV in the MVP, normalizes it into a canonical model, runs deterministic project-control checks, identifies anomalies, links every finding to source evidence, calculates project health, and uses an LLM to explain causes, impacts, and recommended actions.

The product is intentionally not positioned as an ERP replacement, scheduling system, or generic BI dashboard. Its primary value is early detection: identify what a Project Control professional should investigate before a project issue becomes expensive or difficult to reverse.

The MVP must deliver one clear outcome: a user can create a project, upload Budget / Actual Cost / Schedule / Progress datasets, validate the data, run Control Check, and receive a prioritized list of evidence-backed findings within minutes.

1.1 Product Promise

"Give ControlCheck your project data and it will tell you what needs attention, why it matters, and where the evidence is."

1.2 Core Differentiation

- Evidence-first AI: every important statement traces to a calculation and source rows.
- Domain-first rules: project-control logic is deterministic and transparent before LLM reasoning is applied.
- Fast onboarding: Excel-first ingestion minimizes integration barriers.
- Action orientation: findings can become accountable actions rather than ending as dashboard alerts.
- Expandable platform: the same foundation can later ingest SAP, Oracle, Primavera, MS Project, APIs, and other sources.

2. Product Vision & Strategy

2.1 Vision

Become the assurance and early-warning layer for project control teams, helping them detect cost, schedule, progress, and data-quality issues before they become project problems.

2.2 Mission

Reduce the time required to inspect project data while improving the consistency, traceability, and actionability of project-control reviews.

2.3 Product Principles

- Trust over novelty: deterministic calculations and evidence are more important than impressive AI language.
- Smallest sellable product first: solve a narrow, recurring pain before adding enterprise breadth.
- Explain every number: users should be able to inspect how metrics and findings were derived.
- Human-in-the-loop: AI recommends; authorized project staff decide and act.
- Progressive intelligence: data quality → rules → anomaly detection → AI reasoning → predictive capabilities.

3. Problem Statement

3.1 Current State

- Project Control teams frequently work across multiple Excel files and manual monthly reviews.

- Data formats differ between projects, vendors, and reporting cycles.
- Review effort is high and depends heavily on individual analyst experience.
- Problems can be visible in hindsight but not prioritized early enough for management.
- BI dashboards show status but often do not explain what should be investigated next.
- Generic LLMs can summarize files but may hallucinate, miscalculate, or lack traceability.

3.2 Core Pain Points

Manual cross-checking	Hours spent comparing datasets	Automated deterministic checks
Inconsistent Excel schemas	Slow preparation and cleaning	AI-assisted schema mapping + canonical model
Hidden cost leakage	Potential financial exposure	Variance, commitment, transaction, and anomaly rules
Schedule slippage discovered late	Reduced recovery options	Progress / baseline / actual checks
Weak evidence traceability	Low trust in AI outputs	Evidence Engine linking findings to records
Too many alerts	Alert fatigue	Severity + confidence + prioritization
Reporting overhead	Time spent writing monthly reports	AI-assisted executive summary and PDF report

4. Target Market & Personas

4.1 Ideal Customer Profile

Industry	EPC, construction, oil & gas services, selected mining contractors
Company size	50–500 employees
Active projects	5–30 projects
Project value	Typically Rp10B–Rp1T+
Current workflow	Excel-heavy monthly project controls
Buyer	Project Controls Manager / PMO / Operations Director
Economic value	Reduce review time and expose material risks earlier

4.2 Personas

Project Control Engineer	Prepare and review project data	Manual reconciliation	Fast findings with evidence
Project Control Manager	Challenge project performance	Limited time across projects	Portfolio view + prioritization
Project Manager	Understand what needs intervention	Too many reports, too little insight	Root cause + action recommendations
Management / Director	See material exposure and trend	Information is fragmented	Executive health + top risks
Admin / Data Steward	Maintain projects and datasets	Formatting inconsistency	Validation, access, audit trail

5. Goals, Non-Goals & Success Metrics

5.1 MVP Goals

- Support organization and project creation.

- Accept XLSX/CSV project datasets.
- Map source columns to canonical fields.
- Perform data-quality checks before analysis.
- Run a rule library across cost, schedule, progress, and data quality.
- Generate findings with severity, impact, confidence, and evidence.
- Calculate a transparent project health score.
- Provide evidence-backed AI explanations and Q&A.
- Generate an executive report.
- Provide an auditable history of analysis runs.

5.2 Non-Goals for MVP

- No ERP replacement.
- No full scheduling authoring or Primavera editing.
- No automated financial posting.
- No autonomous contract changes or approvals.
- No multi-agent swarm architecture.
- No mobile app requirement.
- No enterprise SSO requirement unless a pilot needs it.
- No predictive EAC model as a release blocker; it is a later enhancement.

5.3 North-Star Metrics

Finding Precision	Validated useful findings / reviewed findings	≥70% initial; >85% after pilot iteration
Time-to-Insight	Upload complete → first findings visible	<5 minutes for standard dataset
Data Mapping Success	Columns mapped without manual custom coding	≥80% for supported templates
Evidence Coverage	Material findings with source evidence	100%
False Positive Rate	Findings rejected by domain reviewer	<30% initial
User Time Saved	Manual review time vs ControlCheck review time	≥50% target
Pilot Activation	Pilot projects completing first analysis	≥80%

6. Product Scope & Feature Matrix

Authentication & Organization	Deferred in Phase 4A	SSO / SCIM	Tenant isolation now; authentication and complete RBAC later
Projects	Yes	Portfolio planning	Core tenant boundary
Data Upload	Yes	ERP/API connectors	XLSX/CSV first
Schema Mapping	Yes	Template learning	AI-assisted suggestions
Data Quality	Yes	Continuous monitoring	Pre-check gate
Cost Control	Yes	Predictive EAC	Rules first
Schedule Control	Yes	Primavera integration	Baseline vs actual
Progress Control	Yes	Forecasting	Plan vs actual
Findings	Yes	Workflow automation	Central intelligence object
Evidence	Yes	Advanced lineage	Critical differentiator
Health Score	Yes	Custom weighting	Transparent formula
AI Assistant	Yes	Agentic workflows	Evidence-grounded
Reports	Yes	Template builder	PDF first
Actions	Yes (basic)	Workflow / reminders	Accountability layer
Billing	Basic / manual	Stripe or local billing	After pilot
Integrations	No	SAP/Oracle/P6/MSP/API	Roadmap

7. User Journeys

7.1 First-Time Project Setup

1. User signs in.
2. Creates or selects organization.
3. Creates project with name, code, project type, currency, start/end date.
4. Uploads first dataset.
5. System detects sheets and candidate schemas.
6. User reviews suggested mappings.
7. System validates data and shows quality summary.
8. User fixes errors or accepts warnings.
9. System stores normalized data and creates an analysis run.

7.2 Run Control Check

1. User clicks Run Control Check.
2. System snapshots the normalized dataset version.
3. Rule Engine executes supported checks.
4. Analytics Engine computes derived metrics.
5. Anomaly detection runs where sufficient data exists.
6. Finding Engine deduplicates and prioritizes findings.
7. Evidence Engine attaches source records and calculations.
8. Health Engine calculates scores.

9. AI Reasoning generates explanations and recommendations for supported high-value findings.
10. Dashboard updates with results and run timestamp.

7.3 Investigate Finding

1. Open Findings.
2. Filter by severity/category/WBS/date.
3. Open finding detail.
4. Review metric and impact.
5. Inspect evidence rows.
6. Read AI explanation.
7. Create or assign action.
8. Mark finding as accepted, rejected, resolved, or needs review.

8. Information Architecture & UX

8.1 Primary Navigation

Dashboard
 Projects
 Data
 Findings
 Cost
 Schedule
 Progress
 AI Assistant
 Reports
 Settings

8.2 Main Screens

Dashboard	Executive project view	Health score, KPI cards, top findings, trend, actions
Projects	Project portfolio	Project cards/table, status, health, last check
Project Overview	Project context	Health, datasets, latest run, top findings
Data Upload	Ingest data	Upload, sheet detection, mapping, validation
Data Quality	Improve trust	Quality score, errors/warnings, row evidence
Findings List	Prioritize issues	Severity filters, category, WBS, impact
Finding Detail	Investigate	Why, impact, evidence, AI, action
Cost	Cost analysis	Budget/actual/variance, drivers, trends
Schedule	Schedule analysis	Progress, delays, baseline vs actual
Progress	Progress control	Planned vs actual, WBS heatmap
AI Assistant	Ask questions	Conversation + evidence references
Reports	Generate/share	Monthly report, PDF export
Settings	Tenant/project controls	Users, roles, thresholds, preferences

8.3 UX Principles

- Critical information above the fold.

- Use red/amber/green only for semantic status, not decoration.
- Every major metric must have a tooltip or formula reference.
- Never hide source evidence behind AI-only prose.
- Keep finding investigation within 1–2 clicks from dashboard.
- Long-running analysis must expose progress and run status.

9. Data Model & Canonical Data Contract

9.1 Core Entities

organizations

■■■■ users

■■■■ projects

■■■■ datasets

■■■■ budget_lines

■■■■ actual_cost_lines

■■■■ schedule_activities

■■■■ progress_snapshots

■■■■ analysis_runs

■■■■ findings

■■■■ evidence_links

■■■■ actions

9.2 Entity Definitions

organizations	Tenant boundary	id, name, status, plan
users	Identity	id, organization_id, role, email, status
projects	Project master	id, organization_id, code, name, currency, dates, status
datasets	Imported source	id, project_id, type, file_name, version, checksum, status
budget_lines	Canonical budget	id, project_id, wbs_code, cost_code, amount, period
actual_cost_lines	Canonical cost transactions	id, project_id, date, wbs_code, cost_code, vendor, po_number, amount, source_row
schedule_activities	Canonical schedule	id, project_id, activity_id, wbs_code, baseline dates, actual dates, progress, status
progress_snapshots	Period progress	id, project_id, period, wbs_code, planned_pct, actual_pct
analysis_runs	Immutable analysis execution	id, project_id, dataset_version, started_at, completed_at, status, engine_version
findings	Primary intelligence object	id, run_id, category, severity, title, impact, confidence, status
evidence_links	Finding traceability	id, finding_id, dataset_id, source_row, record_id, calculation_ref
actions	Remediation workflow	id, finding_id, owner_id, due_date, status, notes

9.3 Canonical Dataset V1

Dataset	Recommended	Fields
Budget	Yes	project_id, wbs_code, cost_code, description, budget_amount
Actual Cost	Yes	transaction_date, project_id, wbs_code, cost_code, vendor, po_number, description, actual_amount
Schedule	Yes	activity_id, wbs_code, activity_name, baseline_start, baseline_finish, actual_start, actual_finish, planned_progress, actual_progress, status
Progress	Recommended	period, wbs_code, planned_progress, actual_progress

9.4 Data Type Rules

- Dates stored as ISO date/time in UTC or project timezone metadata.
- Money stored as numeric decimal; no floating-point persistence for financial values.
- Percentages stored as decimal 0–1 internally and formatted as 0–100% in UI.
- Source row and source file metadata retained for traceability.
- All imported records receive a stable surrogate ID and source checksum.

10. Data Ingestion & Quality Engine

10.1 Ingestion Pipeline

Upload → File Scan → Sheet Detection → Schema Detection → Mapping Suggestions → Validation → Normalization → Quality Score → Ready for Analysis

10.2 Data Quality Rules

Rule	Condition	Severity
Missing required field	WBS missing	Critical if > threshold
Invalid date	Finish < Start	Critical
Invalid percentage	Progress >100%	Warning/Critical
Negative cost	Actual cost <0	Warning
Duplicate transaction	Same vendor/date/PO/amount	Warning
Orphan WBS	Actual WBS absent from budget	Critical/Warning
Unmapped column	Source column not mapped	Warning
Inconsistent vendor	"ABC Ltd" vs "PT ABC"	Warning
Currency inconsistency	USD rows in IDR project	Critical
Schema drift	Columns changed vs prior version	Warning

10.3 Analysis Gate

A project can run analysis with warnings, but critical data-quality errors that make a metric unreliable must block the affected rule family. The UI must state exactly what is blocked and why.

11. Control Rule Engine

11.1 Rule Architecture

Rule Definition → Scope → Input Query → Calculation → Threshold → Severity → Finding Template → Evidence Query

11.2 Rule Object

```
{
  "rule_code": "COST-OVR-001",
  "category": "COST",
  "name": "Actual exceeds budget",
  "version": "1.0",
  "thresholds": {"warning": 0.05, "critical": 0.10},
  "requires": ["budget", "actual_cost"],
  "evidence_fields": ["wbs_code", "source_row"]
}
```

11.3 Initial Rule Families

	Initial Rule Families
Cost	Actual > budget; actual variance; commitment > budget; cost spike; unallocated cost; duplicate payment-like records
Schedule	Overdue activity; baseline/actual slip; planned vs actual progress; critical activity delay
Progress	Actual below plan; progress stagnation; progress >100%; cost rising while progress flat
Data Quality	Missing fields; duplicates; invalid values; orphan WBS; schema drift
Cross-Domain	Cost rising + progress flat; schedule slip + high remaining cost; WBS performance concentration

11.4 Threshold Configuration

Default thresholds ship with the product, but Project Control Managers can configure supported thresholds at organization or project level. Changes are versioned and included in analysis-run metadata.

11.5 Rule and Threshold Governance

Every analysis run is bound to an immutable rule-catalogue version. Each structured runtime definition declares its evaluation grain, operator, thresholds, severity bands, lookback, materiality, and permitted exclusions. Dataset, catalogue, and ground-truth versions must be recorded with the run for reproducibility.

- CST-004 uses the WBS|VENDOR grain so vendor concentration is never ambiguous across work packages.
- CST-005 triggers only when a transaction is at least 25% of WBS budget and at least 3% of project budget.
- PRG-003 applies a current-period materiality floor of 1% of project budget before cost-growth logic is evaluated.
- Threshold or severity changes require a new catalogue version, boundary tests, adjudication rationale, and regression evidence.

11.6 Approved Exceptions

An Approved Exception is an explicit governance record, not silent suppression. It requires an exception ID, rule/entity scope, rationale, approver, effective period, and evidence reference. Expired or unmatched exceptions do not alter findings.

- Raw results always retain the finding and the applicable exception reference.
- Exception-aware quality metrics may exclude approved cases from FP/FN counts while raw metrics remain visible.
- Front-loaded procurement or advance-payment patterns may be approved only through this controlled mechanism.

12. Findings & Evidence Engine

12.1 Severity

Critical	Immediate management attention	High impact, material exposure, or severe control failure
Warning	Needs investigation	Meaningful deviation or emerging risk
Observation	Useful insight	Non-critical anomaly or improvement opportunity

12.2 Finding Lifecycle

NEW → REVIEWING → ACCEPTED / REJECTED / ACTION_CREATED → IN_PROGRESS → RESOLVED

12.3 Evidence Requirement

No material finding should be presented as a confirmed issue without a machine-readable evidence path. For derived findings, the system stores the formula/reference plus the source records used in the calculation. AI-generated narrative may paraphrase, but cannot create unsupported values.

12.4 Finding Priority

A default priority score can combine severity, impact, confidence, recurrence, and age. The formula must remain inspectable and configurable rather than being hidden inside the LLM.

$Priority = 0.35 \times SeverityScore + 0.30 \times ImpactScore + 0.20 \times ConfidenceScore + 0.10 \times RecurrenceScore + 0.05 \times AgeScore$

13. Project Health Scoring

13.1 Initial Model

$Health\ Score = 0.30 \times Cost\ Score + 0.30 \times Schedule\ Score + 0.25 \times Progress\ Score + 0.15 \times Data\ Quality\ Score$

13.2 Score Bands

80–100	Healthy	No material control concern
65–79	Moderate Risk	Management attention recommended
50–64	High Risk	Active intervention likely required
0–49	Critical	Immediate review required

13.3 Transparency Requirement

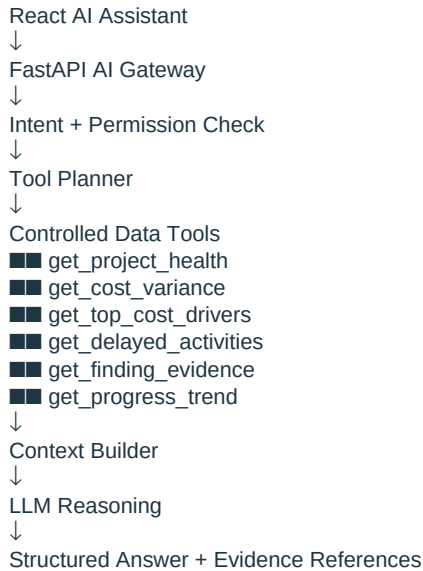
Every score must provide component scores, weights, key drivers, and a “Why this score?” view. Future customer-specific scoring can override weights only through a controlled configuration.

14. AI Intelligence Architecture

14.1 AI Principle

The LLM is a reasoning and communication layer, not the source of truth for calculations. SQL/Python/rules determine facts; the LLM interprets those facts within a constrained context.

14.2 Architecture



14.3 AI Safety Rules

- Never invent financial values, activity IDs, dates, or evidence.
- State when required data is unavailable.
- Use “potential exposure” rather than “loss” unless the evidence establishes an actual loss.
- Do not issue accounting, contractual, or legal decisions autonomously.
- Respect tenant/project access before any tool call.
- Log prompt template version, tool calls, source IDs, and model response metadata for auditability where policy permits.

15. AI Assistant Requirements

15.1 Supported Query Types

Health explanation	Why is health score 68?	get_project_health, top_findings
Cost drivers	What caused the cost variance?	get_cost_variance, get_top_cost_drivers
Schedule risk	Which activities are most at risk?	get_delayed_activities
Finding investigation	Show evidence for WBS 03.02 overrun.	get_finding_evidence
Action planning	What should I do first this week?	get_top_findings, action context
Comparison	Which WBS is worst?	get_wbs_performance

15.2 Response Structure

Answer
Key Evidence
Impact
Recommended Action
Confidence / Data Caveat
Evidence References

15.3 AI Evaluation

A golden-question test set should be maintained from pilot data. Each answer is evaluated for factual correctness, evidence alignment, completeness, and usefulness by a domain reviewer.

16. API Specification

16.1 Authentication (Deferred)

Method	Endpoint	Description
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/refresh	Refresh session
GET	/api/v1/me	Current user

16.2 Projects & Datasets

Method	Endpoint	Description
GET	/api/v1/projects	List projects
POST	/api/v1/projects	Create project
GET	/api/v1/projects/{id}	Project detail
POST	/api/v1/projects/{id}/datasets/upload	Upload dataset
GET	/api/v1/datasets/{id}/mapping	Get mapping suggestions
POST	/api/v1/datasets/{id}/mapping/confirm	Confirm mapping
GET	/api/v1/datasets/{id}/quality	Quality result

16.3 Analysis & Findings

Method	Endpoint	Description
POST	/api/v1/projects/{id}/analysis-runs	Start analysis
GET	/api/v1/analysis-runs/{id}	Run status
GET	/api/v1/projects/{id}/findings	List findings
GET	/api/v1/findings/{id}	Finding detail
GET	/api/v1/findings/{id}/evidence	Evidence records
POST	/api/v1/findings/{id}/actions	Create action
PATCH	/api/v1/actions/{id}	Update action

16.4 AI & Reports

Method	Endpoint	Description
POST	/api/v1/ai/chat	Evidence-grounded assistant
POST	/api/v1/reports/project/{id}/monthly	Generate monthly report
GET	/api/v1/reports/{id}	Report status/download metadata

16.5 API Standards

- JSON response envelope with request_id.
- Validation errors use structured codes.
- Pagination for lists.
- Idempotency keys for uploads and analysis runs.
- OpenAPI generated from FastAPI.
- Every mutating request records actor and timestamp.

17. Security, Multi-Tenancy & Governance

17.1 RBAC (Deferred)

Org Admin	Manage users, projects, settings, plan
Project Control Manager	Create/edit projects, run analysis, review findings, manage actions
Project Control Engineer	Upload data, run analysis, investigate findings
Project Manager	View project findings, create/track actions
Viewer	Read-only dashboards/reports

17.2 Tenant Isolation

- Every business row carries organization_id directly or through controlled project relation.
- Backend authorization checks are mandatory; frontend hiding is not security.
- Object-storage paths are tenant-scoped.
- Export endpoints enforce the same access model.
- Audit events retain actor, object, action, and timestamp.

17.3 Data Governance

- Uploaded source files are immutable by default after successful import; corrections create a new version.
- Analysis runs are immutable snapshots.
- Deletion should support soft-delete plus governed retention.
- Sensitive project data should not be used for model training by default unless the customer explicitly opts in under applicable agreements.

18. Reporting

18.1 Monthly Control Report

Executive Summary	Health, major movements, critical findings
Cost	Budget, actual, variance, top drivers
Schedule	Planned vs actual, delayed activities
Progress	Plan vs actual, WBS exceptions
Data Quality	Quality score, blocking issues
Findings	Top prioritized findings with impact
Actions	Open actions, owners, due dates
AI Commentary	Concise evidence-backed narrative

18.2 Report Rules

- Report values must come from the same analysis-run snapshot shown in the application.
- AI narrative must reference the run context and not current unsnapshotted data.
- PDF must include report date, project code, analysis version, and data period.

19. Technical Architecture

19.1 Application Stack

Frontend	React + TypeScript	Production SaaS UX
UI	Tailwind + shadcn/ui	Consistent enterprise components
Data fetching	TanStack Query	Server-state management
Charts	Recharts / suitable chart library	Fast implementation
Backend	FastAPI + Python	Strong fit for analytics and API
ORM	SQLAlchemy + Alembic	PostgreSQL migrations
Database	PostgreSQL	Multi-tenant transactional foundation
Analytics	Pandas / NumPy / Scikit-learn	Project-control calculations and anomaly detection
File storage	S3-compatible / Supabase Storage	Source files and generated artifacts
Background jobs	Celery/RQ or managed queue later	Long-running analysis
Observability	Structured logs + error monitoring	Pilot reliability

19.2 Service Boundaries for MVP

Keep the codebase modular but deploy as a small number of services. Recommended MVP: one React frontend, one FastAPI application, one worker process for analysis jobs, PostgreSQL, and object storage. Split into independent microservices only when scale or team ownership requires it.

19.3 Processing Model

Web Request
↓
Create Analysis Run
↓
Queue Job
↓
Worker
■ Load normalized data
■ Data quality gate
■ Rule execution
■ Metrics / anomalies
■ Findings + evidence
■ Health score
■ AI enrichment
↓
Persist results
↓
Frontend polling / websocket update

20. MVP Roadmap & Delivery Plan

Sprint 1 — Product & Data Foundation

- Finalize PRD and acceptance criteria.
- Create PostgreSQL schema/migrations.
- Define canonical datasets and sample templates.
- Prepare dummy EPC project dataset.
- Define initial rule catalog.

Sprint 2 — Ingestion & Data Quality

- Project CRUD.
- XLSX/CSV upload.
- Sheet/column detection.
- Mapping UI and mapping persistence.
- Normalization pipeline.
- Data quality engine and UI.

Sprint 3 — Control & Findings

- Complete adjudication, Golden Positive, and Boundary / Negative validation alignment before frontend or AI reasoning expansion.
- Cost rule engine.
- Schedule rule engine.
- Progress rule engine.
- Cross-domain checks.
- Finding object and lifecycle.
- Evidence engine.

Sprint 4 — Health & AI

- Health score.
- Top drivers.
- AI explanation.
- AI Assistant with controlled tools.
- Golden-question evaluation set.

Sprint 5 — Production UX & Reporting

- React dashboard.
- Finding detail.
- Actions.
- Monthly report.
- Audit log basics.
- Performance hardening.

Sprint 6 — Pilot

- Deploy production pilot.
- Onboard 2–3 project teams.
- Measure precision and time saved.
- Fix false positives.
- Prepare pricing and sales demo.

21. QA, Evaluation & Acceptance Criteria

21.1 Test Layers

- Unit tests for every rule calculation.
- Integration tests for ingestion → normalization → finding.
- API contract tests.
- UI smoke tests for core journey.

- Golden dataset regression tests.
- AI evaluation tests for factuality and evidence alignment.

21.2 Key Acceptance Criteria

AC-01	Upload Excel	Supported XLSX uploads without data loss
AC-02	Schema mapping	User can review and confirm mappings
AC-03	Quality gate	Critical issues clearly block affected analyses
AC-04	Rule correctness	Known test cases produce expected values
AC-05	Evidence	Every material finding has evidence reference
AC-06	Health score	Score reproduces from documented formula
AC-07	AI grounding	AI answer cites available evidence and avoids invented numbers
AC-08	Tenant isolation	Cross-organization project access is rejected server-side; full RBAC is deferred
AC-09	Report consistency	PDF values match selected analysis run
AC-10	Performance	Standard pilot dataset returns initial result within agreed target

21.3 Ground-Truth Governance

Ground truth must be curated independently from production rule functions. It is exhaustive for each controlled fixture and stores a unique rule/entity match key, expected severity, metric expectations, evidence anchors, and an adjudication reference. Any label change requires rationale and version history.

- Golden Positive v0.2 contains 59 exhaustive expected findings across all 20 deterministic rules.
- Boundary / Negative v0.2 contains 50 literal below, equal, above, and approved-exception cases across 16 numeric rules.
- Representative or selectively planted labels must not be described as exhaustive ground truth.

21.4 Validation Metrics

Release acceptance requires 100% on controlled validation fixtures for precision, recall, severity accuracy, and metric agreement; zero unreviewed labels; zero unexplained FP/FN; and deterministic repeated output. This is not a customer-accuracy claim and must not be presented as performance on unseen project data.

- Golden Positive v0.2 verified result: 59 TP, 0 FP, 0 FN, and 100% severity and metric agreement.
- Boundary / Negative v0.2 verified result: 0 FP, 0 FN, with all 50 manifest cases reconciled.
- Raw and exception-aware metrics are reported together; approved exceptions remain auditable.

21.5 Artifact Version Compatibility

Before any rule executes, dataset, rule catalogue, and ground truth must share the same major/minor version. A mismatch stops processing, writes no findings output, and returns the stable error code `incompatible_artifact_versions` through CLI and API.

22. Pilot & Go-to-Market

22.1 Pilot Offer

Offer a tightly scoped 30-day pilot to a small number of contractors. The pilot should use one real project per customer and focus on a measurable before/after outcome.

22.2 Pilot Questions

- How long does the team spend preparing data today?
- Which checks are repetitive?

- Which findings are considered useful?
- Which findings are false positives?
- Which findings changed a project decision?
- How much time was saved?
- What value did the customer estimate from early detection?

22.3 Sales Demo Story

1. Upload messy project Excel.
2. Show automated mapping and data-quality issues.
3. Run Control Check.
4. Show health score and top 5 findings.
5. Open one cost finding.
6. Show source rows/evidence.
7. Ask AI why the issue happened.
8. Create an action for the responsible owner.
9. Generate monthly executive report.

23. Pricing Hypothesis

Starter	Rp999K/month	Up to 3 projects, basic findings, PDF
Professional	Rp3.5M/month	Up to 10 projects, AI Assistant, team access
Business	Rp8–15M/month	More projects, controls, reporting, priority support
Enterprise	Custom	Private deployment, SSO, integrations, custom policies

Pricing is a hypothesis for validation, not a fixed market claim. The pilot should test willingness-to-pay against measurable time saved and potential exposure identified.

24. Risks & Mitigations

False positives	Low trust	Domain review, rule tuning, confidence and evidence
Messy data	Failed onboarding	Strong data-quality and mapping layer
AI hallucination	High trust/compliance risk	Tool-based grounding and evidence constraints
Too broad scope	Slow MVP	Hard MVP boundaries
Low customer urgency	Weak sales	Sell measurable early-warning value
Security concerns	Enterprise blockers	Tenant isolation, audit trail, clear data policy
Over-reliance on one LLM	Cost/vendor risk	Provider abstraction in AI gateway
Performance on large files	Poor UX	Async processing and staged profiling

25. Definition of Done

ControlCheck AI MVP is considered ready for pilot when all of the following are true:

- A real user can create an organization and project.

- A real user can upload the four supported dataset types.
- Mapping can be reviewed and confirmed without developer intervention.
- Data quality results are visible and actionable.
- All 20 deterministic rules meet the governed v0.2 controlled-fixture acceptance gates before pilot use.
- Findings are prioritized and have evidence.
- Health score is reproducible and explainable.
- AI Assistant can answer a defined set of questions using controlled tools.
- PDF report is generated consistently from an analysis run.
- Phase 4A tenant predicates prevent cross-organization access; full RBAC is deferred.
- Pilot logging captures analysis failures and AI evaluation data.
- The product can be deployed from a reproducible environment.

Appendix A: Example Rules

A1. COST-OVR-001 — Actual Exceeds Budget

Scope: WBS

Formula: $\text{Variance} = \text{Actual} - \text{Budget}$

Critical if $\text{Variance} / \text{Budget} \geq 10\%$

Warning if $\geq 5\%$ and $< 10\%$

Evidence: budget rows + actual rows

Finding: "Potential cost overrun on {wbs_code}"

A2. COST-SPIKE-001 — Spending Acceleration

Compare current period spending with trailing average or prior comparable period.

Trigger when increase exceeds configured threshold and minimum transaction volume is met.

Do not trigger when dataset coverage is insufficient.

A3. SCHED-SLIP-001 — Baseline Finish Slippage

Trigger when $\text{actual_finish} > \text{baseline_finish}$ OR $\text{current date} > \text{baseline_finish}$ for incomplete activity.

Severity increases for critical activities and larger delay days.

A4. PROG-GAP-001 — Planned vs Actual Progress

Gap = $\text{Planned Progress} - \text{Actual Progress}$

Warning at configured gap threshold; Critical when combined with cost/schedule deterioration.

A5. CROSS-COSTPROG-001 — Cost Rising While Progress Flat

Trigger when cost trend is increasing materially while actual progress remains flat within tolerance.

Recommend review of productivity, quantities, vendor charges, and potential rework.

Appendix B: Example API Payloads

B1. Create Analysis Run

```
{
  "project_id": "proj_123",
  "dataset_ids": ["ds_budget_01", "ds_cost_01", "ds_sched_01", "ds_prog_01"],
  "rule_set_version": "v1.0",
  "options": {"enable_anomaly_detection": true, "enable_ai_enrichment": true}
}
```

B2. Finding Response

```
{
  "id": "finding_001",
  "severity": "CRITICAL",
  "category": "COST",
  "title": "Potential cost overrun on WBS 03.02",
  "metric": {"budget": 1000000000, "actual": 1240000000, "variance": 240000000},
  "confidence": 0.91,
  "impact_type": "POTENTIAL_EXPOSURE",
  "evidence_count": 14,
  "status": "NEW"
}
```

Appendix C: Demo Scenario

Demo project: "EPC Tank Farm Expansion" with a 12-month baseline. The sample package should contain 1–2 thousand actual-cost rows, 100–300 schedule activities, monthly progress, and intentionally seeded control issues. Expected seeded issues include one WBS cost overrun, two overdue activities, one orphan WBS, duplicate transactions, a progress gap, and a cost-versus-progress anomaly.

The demo goal is not to show the largest number of alerts. It is to demonstrate that ControlCheck finds a small number of meaningful issues, explains them clearly, and lets the user inspect the supporting evidence.

Document Change Log

Change Log v0.2

Validation Alignment v0.2 formalizes rule and threshold governance, approved exceptions, exhaustive ground-truth governance, controlled-fixture metrics, artifact compatibility, and the validation-before-UI roadmap gate.

0.1	17 Aug 2026	Initial product requirements baseline	Product / Founder
0.2	17 Aug 2026	Validation Alignment: governed rules, fixtures, ground truth, metrics, exceptions, and version preflight	Product / Founder

Phase 4A Product Alignment

Phase 4A turns the validated deterministic engine into a durable, organization-scoped backend. It preserves the stateless audit endpoint while adding registered projects, uploaded workbook storage, dataset snapshots, reproducible rule catalogue versions, analysis-run history, persisted findings/evidence, lifecycle status updates, and audit events.

Scope delivered in Phase 4A

- PostgreSQL 15+ persistence managed by SQLAlchemy 2 and Alembic migrations.
- Local atomic file-storage adapter with path-safety controls and SHA-256 integrity metadata.
- Organization-scoped project creation/listing using the temporary X-Organization-ID contract.
- Durable analysis execution with Golden and Boundary workbook behavior preserved.
- Run history, finding filters, detailed evidence retrieval, and governed finding status transitions.
- Stable API error envelope and request ID for operational traceability.

Explicit deferrals and sequencing

Authentication and complete RBAC are deferred to a later hardening phase. The temporary organization header is suitable only for controlled development and validation environments, not public production exposure.

Phase 4B will add raw-row lineage and canonical WBS, budget, cost, commitment, schedule, and progress facts. This preserves the Phase 4A delivery focus while preparing database-native analytics and richer exception processing.

Frontend implementation remains outside the current phase. The backend contracts should stabilize before UI workflows are built.

Acceptance criteria added in v0.3

- Golden workbook creates one succeeded run with 59 findings and source-traceable evidence.
- Boundary workbook creates one succeeded run with zero findings.
- Workbook/project identity mismatch creates no analysis run.
- Engine failure creates one failed run and no findings.
- Cross-organization access returns not found and never leaks tenant records.
- Alembic upgrade, downgrade, re-upgrade, and zero-drift checks pass against PostgreSQL.
- All versioned historical reference artifacts remain byte-for-byte unchanged.

Change Log

0.1	17 Aug 2026	Initial MVP blueprint.
0.2	17 Aug 2026	Rule and synthetic validation alignment.
0.3	17 Aug 2026	Phase 4A PostgreSQL persistence, durable API, storage, lifecycle, and Phase 4B sequencing.